Описание проекта: «Дашборд результатов экспериментов»

Тема направления — «Анализ данных». Веб-приложение для исследования: учёт, поиск и сравнение ML-экспериментов (Next.js + FastAPI + PostgreSQL + Docker + GitHub Actions). Адреса: фронтенд http://localhost:3000, API http://localhost:8000, документация API http://localhost:8000/docs. Запуск: docker compose up --build.

1. Задача, пользователь, связь с диссертацией

Приложение решает задачу исследователя по анализу данных: фиксировать каждый запуск (датасет, модель, статус, accuracy, длительность), искать по названию/датасету/модели, фильтровать по статусу, сортировать и смотреть сводную статистику дашборда. Пользователь — докторант/исследователь, ведущий серию экспериментов. Связь с диссертацией: воспроизводимость и сравнимость результатов экспериментов; данные — только синтетические, персональных данных нет.

2. Главная сущность Experiment

Поле	Тип	Обязательность	Ограничения
id	integer, PK	авто	primary key
name	string(500)	да	мин. 3 символа, indexed
dataset	string(500)	да	мин. 2 символа, indexed
model_name	string(500)	да	мин. 2 символа, indexed
status	string(50)	да	planned | running | completed | failed, default planned
accuracy	float | null	нет	0 … 1
duration_sec	float | null	нет	>= 0
notes	text | null	нет	—
created_at	datetime tz	авто	server_default now()

3. Функциональные требования (разд. 2.1) — выполнение

Требование	Реализация
1. Список с сортировкой и пагинацией	GET /experiments (limit/offset, sort_by, sort_order); стр. /experiments с пагинацией
2. Карточка записи	GET /experiments/{id}; стр. /experiments/[id]
3. Добавление через форму с проверкой	POST /experiments (Pydantic 422); стр. /experiments/new + Server Action
4. Редактирование и удаление	PATCH /experiments/{id}, DELETE 204; форма в details + кнопка Удалить
5. Поиск минимум по 2 полям	q ищет по 3 полям: name, dataset, model_name (ilike)
6. Документация API /docs	FastAPI Swagger: http://localhost:8000/docs

4. Нефункциональные требования (разд. 2.2) — выполнение

- 1. Запуск одной командой: docker compose up --build (проверено 06.10.2026, все 3 сервиса Up).
- 2. Схема БД только через миграции Alembic (001_create_experiments.py); старт бэкенда выполняет alembic upgrade head.
- 3. Бэкенд покрыт тестами: 10 тестов pytest (CRUD, поиск, фильтр, пагинация, сортировка, stats, 404, 422).
- 4. CI: backend.yml (uv sync, ruff, pytest, docker build) и frontend.yml (npm ci, lint, build); dependabot.yml.
- 5. Секреты не в репозитории: только .env.example; .env в .gitignore.
- 6. Адаптивность: Tailwind, max-w-4xl, сетка stats grid-cols-2 sm:grid-cols-4, формы flex-col sm:flex-row.
- 7. README позволяет запуск без помощи: Быстрый старт, локальная разработка, тесты, структура.

5. Архитектура

Модульный монолит: один бэкенд, одна БД, код разделён на модули (experiments). Микросервисы/Kubernetes не используются.

Компонент	Технология	Роль
Фронтенд	Next.js 15, TypeScript, Tailwind	Страницы, формы, дашборд
Бэкенд	Python 3.12, FastAPI, Pydantic	Логика, валидация, REST API
Доступ к БД	SQLAlchemy 2, Alembic	Модели и миграции
База данных	PostgreSQL 17 + pgvector	Хранение данных
Инфраструктура	Docker, Docker Compose	Одинаковый запуск
CI	GitHub Actions	Тесты и сборка

Схема (Mermaid, см. docs/architecture.md): flowchart LR: Браузер → Next.js :3000 → FastAPI :8000 → PostgreSQL :5432.

Прохождение запроса

- 1. Браузер запрашивает /experiments у Next.js.
- 2. Серверный компонент вызывает GET /experiments у FastAPI.
- 3. FastAPI валидирует параметры (Pydantic), вызывает service.
- 4. Service через SQLAlchemy выполняет SQL к PostgreSQL.
- 5. FastAPI возвращает JSON, Next.js рендерит HTML. Формы идут через Server Actions (POST/PATCH/DELETE).

Слои бэкенда

Файл	Отвечает за	Не должен
models.py	Таблицы БД (SQLAlchemy)	Логики HTTP
schemas.py	Формат API (Pydantic)	Обращаться к БД
service.py	Бизнес-логика и запросы	Знать HTTP-коды
router.py	Эндпоинты и коды ответов	Писать SQL напрямую

6. API эндпоинты

Метод	Путь	Описание	Коды
GET	/health	Проверка	200

Метод	Путь	Описание	Коды
GET	/experiments	Список: q, status, limit, offset, sort_by, sort_order	200, 422
GET	/experiments/stats	Агрегаты дашборда	200
GET	/experiments/{id}	Карточка	200, 404
POST	/experiments	Создать	201, 422
PATCH	/experiments/{id}	Обновить	200, 404, 422
DELETE	/experiments/{id}	Удалить	204, 404

Важно: /stats объявлен выше /{id}, иначе слово stats трактуется как идентификатор.

7. Ключевые архитектурные решения

- 1. Модульный монолит вместо микросервисов — масштаб проекта не требует распределённости.
- 2. FastAPI, а не Django — лёгкий REST и автодокументация /docs.
- 3. Миграции Alembic — единственное изменение схемы; автопрогон при старте контейнера.
- 4. SQLite в тестах — быстро, без Postgres; в проде — PostgreSQL.
- 5. Server Components + Server Actions — данные на сервере, интерактив только в форме.

8. Быстрый старт

PowerShell:

```
cd "C:\Users\tsay.glad\Desktop\9router\gemini"; docker compose up --build
```

CMD:

```
cd /d C:\Users\tsay.glad\Desktop\9router\gemini && docker compose up --build
```

Seed (второй терминал): docker compose exec backend python -m scripts.seed. Адреса: http://localhost:3000/experiments и http://localhost:8000/docs.

9. Локальная разработка без Docker

Бэкенд: cd backend; uv sync; uv run alembic upgrade head; uv run uvicorn app.main:app --reload.
Фронтенд: cd frontend; npm install; npm run dev (API_URL=http://localhost:8000 из .env.local).

10. Тесты

10 тестов, все зелёные (uv run pytest -q), ruff чист. Покрыты: health, create+read, поиск по 2 полям, фильтр status, пагинация+сортировка, stats, update, delete+404, validation 422, 404.
Линтеры и сборки: uv run ruff check ., npm run lint (0 ошибок), npm run build (6 роутов).

11. Структура проекта

Корень: README.md, LICENSE, CITATION.cff, .env.example, docker-compose.yml, docker-compose.prod.yml, Caddyfile, .github/workflows (backend.yml, frontend.yml). backend/: Dockerfile, pyproject.toml, uv.lock, alembic/, app/ (main.py, db.py, core/config.py, modules/experiments/*), scripts/seed.py, tests/. frontend/: Dockerfile, next.config.ts (standalone), src/app (page, experiments, experiments/new, experiments/[id]), src/components/ExperimentForm.tsx, src/lib (api.ts, types.ts). docs/: architecture.md, project-description.pdf.

12. Данные и источник

Только синтетические данные (24 записи seed): BERT/XGBoost/LSTM/ResNet/YOLO и др. на условных датасетах (news-kz-10k, churn-5k и т.п.). Распределение: completed 16, running 3, planned 3, failed 2; средняя accuracy 0.775. Персональных данных нет.

13. CI и развёртывание

Backend CI: uv sync → ruff → pytest → docker build. Frontend CI: npm ci → lint → build. Dependabot: uv/npm/docker/actions. Прод: docker-compose.prod.yml (сброс портов, restart, Caddy :80/:443) + Caddyfile (reverse_proxy frontend:3000, backend:8000).

14. AI assistance

При разработке использовался ИИ-ассистент: генерация кода, тестов, фронтенда и документации по методичке; проверки выполнены запуском pytest, ruff, next build и docker compose up --build с ручной проверкой API и страниц.

15. Лицензия, цитирование, контакты

Лицензия MIT (LICENSE). Цитирование: CITATION.cff (Experiment Dashboard, 0.1.0). Скриншоты: docs/screenshots/ (добавить 2–4 шт. перед защитой). Видео-демо 3–5 минут — записать отдельно.

16. Соответствие критериям (разд. 17)

Критерий	Баллы	Статус
1. Тема + seed-данные	5	выполнено
2. Слои models/schemas/service/router	10	выполнено
3. CRUD + поиск + пагинация + коды	15	выполнено
4. Модели и миграции Alembic	10	выполнено
5. Тесты (10) + линтер	10	выполнено
6. Фронтенд: список/карточка/CRUD	15	выполнено
7. Ошибки + адаптив	5	выполнено
8. Запуск одной командой	10	выполнено, проверено
9. CI + Dependabot + без секретов	10	выполнено
10. README + architecture + скриншоты	5	частично: нужны скриншоты
11. Защита и демо	5	подготовить демо